



Estudiantes:

Owen Torres Porras

Carné:

2023302034

Curso:

Compiladores e Intérpretes

GR 60

Semestre II 2026

Tarea 1

Profesor:

Allan Rodriguez Dávila

Descripción del problema

Un grupo de desarrolladores desea crear un nuevo lenguaje imperativo, ligero, que le permita realizar operaciones básicas para la configuración de chips, ya que esta es una industria que sigue creciendo constantemente, y cada vez estos chips necesitan ser configurados por lenguajes más ligeros y potentes. Es por esto que este grupo de desarrolladores requiere desarrollar su propio lenguaje para el desarrollo de sistemas empotrados, y como primer paso necesitan desarrollar una gramática simple y poderosa. Para ello se requiere desarrollar una gramática BNF para un lenguaje imperativo

Diseño del Programa

No terminales

No terminal	Comentario asociado
<programa>	Inicio del programa, puede haber funciones antes o después de `principal`, pero solo un `principal`.
<funciones>	Forma recursiva para que existan varias funciones o ninguna.
<funcion>	El tipo string se va a restringir de forma semántica.
<main>	Función principal del programa, no recibe parámetros.
<parametros>	Forma recursiva de parámetros separados por comas.
<parametro>	Parámetro simple.
<bloque>	Bloque que encapsula el contenido del programa.
<sentencias>	Forma recursiva para que existan varias sentencias o ninguna.
<sentencia>	La sentencia puede crear o asignar una variable, convertirse en una expresión, llamar una función, transformarse en funciones reservadas o estructura de control.
<creacion>	Para construir variable o arreglos unidimensionales.
<creacionVariable>	Creación con o sin asignación.
<creacionArreglo>	Los tipos de arreglo van a ser validados semánticamente y se permite que sus índices sean expresiones ya que es permitido en lenguajes imperativos siempre y cuando sean expresiones aritméticas, por lo que se validará de forma semántica también.
<valores>	Todos los valores que se asignan al arreglo; se validará que la cantidad sea la correcta en el futuro.
<valor>	Se permite que el valor ingresado sea expresión siempre y cuando sea expresión aritmética; se va a validar a futuro.

<asignacion>	Para modificar un valor existente o asignar un valor a una variable vacía.
<accesoArreglo>	Para acceder al valor que está en algún índice del arreglo; se va a validar semánticamente que el índice sea válido.
<expresion>	La expresión puede llegar a todos los tipos de operaciones y literales.
<expresionAritmetica>	Se va a respetar la precedencia en el sintáctico.
<modulo>	Se va a respetar la precedencia en el sintáctico.
<multiplicacion>	Se va a respetar la precedencia en el sintáctico.
<sumaResta>	Se va a respetar la precedencia en el sintáctico.
<potencia>	Se va a respetar la precedencia en el sintáctico.
<unario>	Se elimina la posibilidad de literal entero negativo para evitar alguna posible ambigüedad en CUP.
<posfijo>	Se respeta que el incremento o decremento vayan después del operando.
<expresionRelacional>	Forma `f:operando1:~ operador f:operando2:~`.
<operandoRelacional>	Forma `f:operando:~`.
<valorRelacional>	Lo que puede ser el operandoRelacional; las llamadas, id o literales son válidas si son enteros o flotantes, que se van a validar semánticamente.
<operadorRelacional>	Ya que no hay símbolos relacionales específicos, se usan los comunes en el paradigma imperativo.
<expresionLogica>	Se separa disyunción y conjunción de negación ya que la negación es unaria. Forma `e:operando1:~ operador e:operando2:~`.
<operadorLogico>	Se usa $\wedge$ y $\vee$.
<operandoLogico>	Forma `e:operando1:~`.
<valorLogico>	Aquí se usa directamente booleano para no validar semánticamente, y tanto id como las llamadas sí se van a validar que sean booleanas.
<parentesisExpresion>	Expresión para colocar paréntesis en operaciones aritméticas.
<operando>	En caso que se necesiten valores sueltos en las expresiones.
<llamadaFuncion>	El id de las funciones va a tener el tipo que se le asignó al crear la función.
<argumentos>	Forma recursiva de argumentos con posibilidad de que sea vacío, separados por comas.
<argumento>	Todo lo que puede recibir como parámetro una función; se va a validar semánticamente que el tipo de parámetro que reciba coincida con el tipo de parámetro esperado.
<estructuraControl>	Aquí se separa en los tres tipos de estructuras de control.
<if>	Puede ser if solo, con uno o más elif, con else o con todos.
<elif>	Permite 0 o varios elif.
<else>	Permite un único else o ninguno.
<while>	Un while simple.
<for>	No hay una forma establecida de for, por lo que se toma inspiración de lenguajes imperativos comunes; se usa `»` como separador.
<condicion>	Condiciones para una estructura de control.
<incrementoDecremento>	Sujeto a cambios por posible ambigüedad.

<return>	Solo retorna char, booleano, entero o flotante; se va a validar semánticamente que no pueda retornar string.
<valorRetorno>	Todos son char, booleano, entero o flotante; string será restringido semánticamente.
<read>	Solo permite leer enteros, flotantes o char, por lo que se van a hacer las validaciones pertinentes.
<write>	Solo permite string, booleano, entero o flotante; se restringirá char.
<valorWrite>	Se restringirá char.
<literal>	Valores literales.

Terminales

Terminal	Valor
<tipo>	"int" "float" "bool" "char" "string"
<char>	"" ([a-zA-Z0-9_!@#%&*=]) ""
<string>	" <caracteres> "
<caracteres>	<caracteres> <caracter> <caracter>
<caracter>	[a-zA-Z0-9_!@#%&*=]
<id>	[a-zA-Z_] [a-zA-Z_] <caracteres>
<booleano>	"true" "false"
<flotante>	<entero> "." <numeros>
<entero>	<numerosPos> <numeros> <numero>
<numeros>	<numeros> <numero> <numero>

Análisis de Resultados

- Se permiten funciones secuenciales con estructuras de control, bloques y sentencias
- Se va permitir usar variables globales pero no hay palabra reservada para globales que las diferencie.
- Se manejan variables, enteras, flotantes, arreglos estáticos unidimensionales, booleanos, caracteres y strings.
- Se permite crear arreglos unidimensionales de tipo flotante y entero pero aún no se valida si es de algún otro tipo pero se validará en el semántico. La cantidad de parámetros también se va a validar en el semántico.

- e. Se permiten sentencias para crear variables con y sin asignación, usando expresiones o literales.
- f. Las expresiones si permiten combinar literales, variables, arreglos y funciones
- g. Se permiten paréntesis y operadores y operandos.
- h. Se permiten expresiones aritméticas de suma, resta, división, potencia y módulo.
- i. Se permiten expresiones unarias de negativo, incremento y decremento.
- j. Se permiten expresiones relacionales sobre enteros y flotantes en la estructura correcta, falta la validación semántica.
- k. Se permiten expresiones lógicas de conjunción, disyunción y negación en la estructura solicitada.
- l. Se permiten sentencias de código que terminan con la comilla latina y se pueden usar en estructuras de control.
- m. Se permite la combinación de expresiones aritméticas, relacionales y lógicas según las reglas del paradigma imperativo.
- n. Se mantiene un tipado fuerte y explícito.
- o. Se permiten estructuras de control if, if-else, if-elif-else, while y for.
- p. Se establecen palabras reservadas write y read que deben ser validadas en tipo durante el análisis semántico.
- q. Se permite la creación de funciones capaces de recibir parámetros que retornan entero, flotantes, char y booleanos. El string será restringido en el semántico.
- r. Se define la estructura de main con el nombre principal para ser la base del programa.
- s. Los comentarios se saltaron por que van a ser contruidos en el análisis léxico.

Bitácora:

<https://github.com/OwenTP27/Tarea1Compiladores.git>